

코딩 작성 가이드

(version 1.0)

작성일자 : 2024-09-30

작성자 : 조창제

목 차

1. 스타일 가이드	1
1). 명칭에 따른 분류	1
2). 괄호 위치에 따른 분류	1
3). 적용안 예시	2
2. 함수 작성	3
1). 함수 작성 요령	3
3. 주석	3
1). 작성 요령	3
2). 주석이 필요한 내용	3
3). 주석이 필요 없는 내용	3
4. 보안	4
1). 민감한 정보 관리	4
5. 협업 규칙	4
6. 문서화	5
1). Commit 가이드	6
7. 메뉴얼 변경이력	7

1. 스타일 가이드(주요 표기 방법)

1). 명칭에 따른 표기법 분류(변수/함수)

(1). 카멜 표기법(Camel Case)

- 정의 : 여러 단어를 하나의 식별자로 표기할 때, 각 단어의 첫 글자를 대문자로 표기하고 나머지는 소문자로 표기하는 방법
- 예시 : camelCaseExample
- 활용 : 주로 변수명, 함수명에 사용

(2). 스네이크 표기법(Snake Case)

- 정의 : 여러 단어를 언더스코어로 구분하여 표기하는 방법
- 예시 : snake_case_example
- 활용 : 주로 변수명, 파일명에 사용

(3). 파스칼 표기법(Pascal Case)

- 정의 : 카멜 표기법과 유사하지만, 첫 글자도 대문자로 표기
- 예시 : PascalCaseExample
- 활용 : 클래스명, 생성자명에 사용

(4). 스크래밍 스네이크 표기법(Screaming Snake Case)

- 정의 : 여러 단어를 언더스코어로 구분하고 문자 모두 대문자로 표기하는 방법
- 예시 : SCREAMING_SNAKE_CASE
- 활용 : 상수나 환경변수 정의에 사용

(5). 케밥 표기법(Kebab Case)

- 정의 : 여러 단어를 -로 구분하고 문자를 모두 소문자로 표기하는 방법
- 예시 : kebab-case-example
- 활용 : 파일명, CSS명 일부

(6). 헝가리안 표기법(Hungarian Notation)

- 정의 : 접두어에 자료형을 붙여 표기하는 방법
- 예시 : strHungarianNotation

2). 괄호 위치에 따른 분류

- K&R

K&R	<pre>if (...) { 처리1(); 처리2(); }</pre>	한 눈에 많은 코드를 볼 수 있음 수평으로 많은 코드를 작성할 수 있음
BSD	<pre>if (...) { 처리1(); 처리2(); }</pre>	블록의 소속을 분명히 할 수 있음 수평으로 많은 코드를 작성할 수 있음
GNU	<pre>if (...) { 처리1(); 처리2(); }</pre>	블록을 if 문 아래 작성함으로 블록의 소속을 분명히 표시

3). 적용안 예시

- 변수명 및 파일명은 스네이크 표기법으로 작성
- 함수나 Method는 카멜 표기법으로 작성
- Class명은 파스칼 표기법으로 작성
- K & R 방식 사용

(7). 작성 조건

- 탭 대신 공백 4개 사용
- 코드라인 길이는 79자를 넘기지 말 것(PEP8 기준)
- 변수 대입 시 등호 양쪽에 공백 1개 기입

(8). 규약

- 보호해야하는 인스턴스는 변수명을 _ 로 시작
- private 인스턴스는 __로 시작
- 모든 import는 파일 맨 위에 작성
- 패키지 import 후 개인 함수 import 작성
- 패키지 import 후 개인 함수 import 시에는 줄바꿈

<pre>import pandas as pd import numpy as np import my_utils</pre>	(X)
<pre>import pandas as pd import numpy as np import my_utils</pre>	(O)

2. 함수 작성

4). 함수 작성 요령

- 함수는 한 가지 작업만 하도록 작성
- 함수의 길이와 복잡성 제한
- 예외처리 및 테스트 코드 작성

3. 주석

5). 작성 요령

- 주석이 한줄인 경우 `//` 로 작성, 여러줄인 경우 `/* */` 활용(python은 `"""` 활용)
- Docstring 활용

6). 주석이 필요한 내용

- 법적 내용 고지
- 의도 설명
- 향후 해야할 일(TODO)

7). 주석이 필요 없는 내용

- 코드와 같은 내용
- 변경이력(git 과 같은 코드 관리시스템을 활용할 것)

법적 내용 고지	// Copyright (C) 2003, 2004, 2005 by Object Montor, Inc. All right reserved. // GNU General Public License
----------	---

4. 보안

8). 민감한 정보 관리

- API키 및 비밀번호는 가급적 환경변수로 관리하되 부득이한 경우 파일로 관리
- AES 256과 같은 암호화 방법 적용

5. 협업 규칙

- 코드 소유권은 코드 작성자와 코드 변경자로 기재
- 프로젝트 관리 툴 사용 시 해당 양식에 맞는 commit 작성

6. 문서화

- 함수, 클래스 사용법 및 설명 문서 작성
- API 문서 작성
- README.md 작성
- README.md에는 프로젝트 설명, 설치 방법, 사용법 기재

제목	<code># Project Name</code> A brief description of what this project does and who it's for.
프로젝트 상태 배지 (버전, 라이선스 등)	<code>![Build Status](https://img.shields.io/badge/build-passing-brightgreen)</code> <code>![Coverage](https://img.shields.io/badge/coverage-90%25-brightgreen)</code> <code>![Version](https://img.shields.io/badge/version-1.0.0-blue)</code>
소개 (프로젝트 목적과 개요)	<code>## Introduction</code> This project is designed to solve [problem] by providing [solution]. It allows users to [main features].
프로젝트 기능	<code>## Features</code> - Feature 1: Description of feature 1. - Feature 2: Description of feature 2. - Feature 3: Description of feature 3.
설치 방법	<code>## Installation</code> To install this project, follow these steps: 1. Clone the repository: ``bash git clone https://github.com/username/project-name.git `` 2. Navigate into the project directory: ``bash cd project-name `` 3. Install dependencies: ``bash npm install `` 4. Start the project: ``bash npm start ``
사용방법	<code>## Usage</code> To use this project, run the following command: ``bash npm run start ``
라이선스	<code>## License</code> This project is licensed under the MIT License - see the [LICENSE](./LICENSE) file for details.
저자	<code>## Authors</code> - Author Name - [GitHub Profile](https://github.com/username)
참고자료	<code>## Acknowledgments</code> - Inspiration: [Project Inspiration](https://inspiration.com) - Special thanks to [Contributor Name](https://github.com/contributor)

9). Commit 가이드

- 하나의 Commit은 하나의 작업이나 변경 사항을 반영하여야 함
- 여러 변경 사항은 하나에 Commit에 담는 것을 가급적 피할 것
- 수정, 변경, 업데이트는 구체적 변경사항 명시

(9). 제목

- 50자 이내로 작성
- 대문자로 시작, 마침표는 사용하지 않음
- 현재형 동사 사용 - ADD, FIX, UPDATE
- 변경 사항의 요약 명확하게 작성

(1) 메시지 유형

- feat : 새로운 기능 추가
- fix : 버그 수정
- docs : 문서 수정
- style : 코드 스타일 변경(포매팅, 세미콜론 누락 등)
- refactor : 코드 리팩토링 (기능 변경 없이 코드 개선)
- test : 테스트 코드 추가 또는 수정
- chore : 빌드 관련 작업, 패키지 매니저 설정 등(코드 변경 없음)

```
feat: add user profile page
fix: resolve NPE in authentication service
docs: update README with installation steps
refactor: simplify user authentication logic
test: add unit tests for payment service
chore: update npm dependencies
```

(10). 본문

- 제목으로 부족한 경우 상세 내용 기재(72자 이내 줄바꿈 사용)
- 무엇을 왜 변경하였는지 작성
- 어떻게 변경하였는지는 코드로 확인이 가능하므로 생략
- 특정 문제나 버그 해결 시 이슈번호를 연결

7